BATCH Batch Job Subsystem

Programmer’s Guide

BATCH-PG-0101

BATCH Version 1.0.1 *** 29-JUL-2026

Contents

Preface **3**
Intended Audience . . . 3
Document Structure . . . 4
Conventions . . . 5

Chapter 1. Concept **6**
1.1 What the Subsystem Is . . . 6
1.2 Decisions and Why . . . 7
1.3 What Is Deliberately Not Done . . . 8

Chapter 2. Architecture **9**
2.1 The Two Images . . . 9
2.2 The Queue . . . 10
2.3 The Path of a Request . . . 11
2.4 The Main Loop . . . 12

Chapter 3. The Modules **13**
3.1 Map . . . 13
3.2 batchdef.h . . . 14
3.3 batchmsg.h and batchmsg.c . . . 15
3.4 batchipc.h . . . 16
3.5 util.c . . . 17
3.6 batchd.c . . . 18
3.7 batch.c . . . 19
3.8 The Vendored Subproject . . . 20

Chapter 4. Data Structures **21**
4.1 BATCH$JOB . . . 21
4.2 BATCHD$JOBENT and BATCHD$CONN . . . 22
4.3 The Protocol . . . 23
4.4 The Journal . . . 24

Chapter 5. Conventions **25**
5.1 Naming . . . 26
5.2 Formatting . . . 27
5.3 The Calling Convention . . . 28
5.4 Diagnostics . . . 29
5.5 Formatting of Output . . . 30

Chapter 6. Making a Change **31**
6.1 Adding a Qualifier . . . 31
6.2 Adding a Condition . . . 32
6.3 Adding a Command . . . 33
6.4 Changing the Job Descriptor . . . 34
6.5 Adding a Check to the Test . . . 35

Chapter 7. The Build System **36**
7.1 Layout 36
7.2 Compilation Options 37
7.3 No Dollar in a CMake File 38
7.4 One Awkwardness 39

Chapter 8. Testing **40**
8.1 What Is There 40
8.2 What the Smoke Test Guards 41

Appendix A. Known Limitations **42**

Preface

Intended Audience

This manual is for the programmer who is to alter the subsystem: to add a qualifier, a message or a command, to correct a defect, or to satisfy himself that a change is safe. It assumes C, POSIX and a reading acquaintance with CMake. It does not assume OpenVMS, although the reader who knows it will recognize why much of the design looks as it does.

The *User's Guide* describes what the subsystem does. This manual describes how it is built, and neither repeats the other.

Document Structure

Part	Contents
Chapter 1	What the subsystem is and the decisions it rests upon
Chapter 2	Architecture: the two images, the queue, the path of a request
Chapter 3	The modules, one by one, and what each routine is for
Chapter 4	The data structures and the two formats that outlive a process
Chapter 5	The conventions the sources hold to, and why
Chapter 6	How to make the changes that come up most often
Chapter 7	The build system
Chapter 8	The tests and what they are guarding
Appendix A	Known limitations and where each of them lives

Conventions

Line numbers are not quoted: they go stale. A routine is named instead, and every routine carries a `FUNCTIONAL DESCRIPTION` block at its head that says what it does, what it takes and what it returns.

Chapter 1. Concept

1.1 What the Subsystem Is

BATCH executes shell scripts as batch jobs: a script is placed in a queue and run later, without a terminal, under a resident job controller. It is a functional analogue of the OpenVMS batch job queue — `SUBMIT`, `SHOW QUEUE`, `DELETE /ENTRY` — and follows that model deliberately rather than the model of `cron` or of a service supervisor.

Three properties follow from the model and shape everything else:

- **A job is identified before it exists.** Submission assigns a queue entry number and returns; the process is created later, if and when the controller decides. The entry, not the process, is what the user holds on to.
- **The queue has states.** A job may wait for an execution slot or for an appointed time. The controller, not the caller, decides when it runs.
- **A job that fails stays.** Its entry and its log survive so they can be looked at. A job that succeeds leaves the queue. This is what makes `SHOW` without qualifiers useful: what remains is what is running and what went wrong.

1.2 Decisions and Why

Two images, not one. `batchd` owns the queue; `batch` is a client that owns nothing. The alternative — a `setuid` utility manipulating a shared file — would put the invariants of the queue in the hands of every caller.

Identification from the kernel. The controller takes the identity of a client from `SO_PEERCRED`, never from the contents of a request. A client can therefore lie about anything except who it is, and everything that matters is checked against that.

One controller, single-threaded. The queue is not locked because nothing else touches it. This is a real constraint on what may be added: anything that blocks the main loop stops the whole subsystem, which is why connections are served without blocking (Section 2.4).

Fixed-size descriptor, no marshalling. `BATCH$JOB` is flat and of fixed size, so it is written to the journal and put on the socket as it stands. The cost is that mostly-empty records are moved about; the gain is that there is no marshalling code to get wrong, and that the journal is readable with `od`. Section 4.4 states the consequence for compatibility.

Condition values, not error codes. Every routine returns an OpenVMS condition value: facility, message number, severity. The low bit is the success indicator, so an outcome is tested without regard to which condition it carries, and the cause is told apart with `$MATCH_COND` rather than by comparing text.

Someone else's library. Logging, messages, the CLI, the queue primitives and the FAO engine come from the StarLet package. The subsystem contributes no general-purpose code of its own, and where the library was found wanting the correction went into the library.

1.3 What Is Deliberately Not Done

- **The subsystem does not stop a job it no longer manages.** A job is a session of its own. When the controller stops, running jobs continue; on the next start they are marked orphaned, or requeued if they carried `/RESTART`. `/RESTART` promises that a job will be run, not that it will be run once.
- **The subsystem holds no configuration file.** Settings come from the command line of the controller and from the build. There is no state to parse and none to get out of step.
- **The subsystem does not manage users.** It has no accounts, no quotas and no access lists. A job runs as whoever submitted it; access to an entry is decided by ownership alone.

Chapter 2. Architecture

2.1 The Two Images

```
   batch (utility)                        batchd (job controller)
   ---------------                        -----------------------
   parses the command line                owns the queue
   builds a request                       schedules jobs
   opens the socket        --- AF_UNIX --> creates processes
   sends, waits, displays  <-- reply -----  collects their status
                                          writes the journal
```

`batch` holds no state between invocations. Everything that outlives a command is in the controller and in the two files described in Section 4.4.

2.2 The Queue

The queue is a doubly-linked list of `BATCHD$JOBENT`, held in a `__QUEUE` of the library. Each entry wraps one `BATCH$JOB` descriptor. An entry is allocated when a job is queued and released when its entry leaves, so the controller holds what its jobs need rather than a table sized for the worst case.

Traversal is by the links directly — the library offers removal from either end but no iterator — and two macros in `batchd.c` say so plainly:

```
#define $JOBQUE_FIRST()     ((BATCHD$JOBENT *) g_jobque.head)
#define $JOBQUE_NEXT(ent)   ((BATCHD$JOBENT *) (ent)->je_links.right)
```

`BATCH_K_MAXJOB` is a ceiling, not a capacity: it exists so that a runaway submitter cannot exhaust memory, and the condition it raises, `BATCH$_NOSLOT`, is more use to the submitter than a failed allocation would be.

The states a job passes through are stated in the *User's Guide*, Section 1.5. In the sources they are the `JOB$K_*` constants of `batchdef.h`. The value zero is unassigned: an entry in the queue always has a state, and an entry that has none does not exist.

2.3 The Path of a Request

```
batch                     batchd
-----                     ------
connect ----------------> s_batchd$conn_accept
                            SO_PEERCRED, O_NONBLOCK, queued on g_connque
send header ------------> s_batchd$conn_read     state CONN$K_RDHDR
send payload -----------> s_batchd$conn_read     state CONN$K_RDPLD
                          s_batchd$conn_dispatch
                            s_batchd$do_submit / do_delete / do_show / release
                            reply assembled in memory
receive reply <---------- s_batchd$conn_write    state CONN$K_WRRSP
                          s_batchd$conn_close
```

The reply is assembled whole before any of it is transmitted. This is what keeps the queue from being held open across a transfer to a slow reader: what the client receives is the queue as it stood when it asked.

2.4 The Main Loop

`main` in `batchd.c` is one loop, and everything the controller does happens in it:

```
reap terminated jobs        s_batchd$reap
schedule what may run       s_batchd$schedule
write the journal if either changed anything
rewind the controller log if it has grown
sweep the job logs, once an hour
build the watched set: listening socket, self-pipe, every open connection
poll
drain the self-pipe
advance each ready connection by one step
accept a new connection
reschedule and journal if a request changed the queue
```

Two properties are load-bearing.

Nothing blocks. Every descriptor is non-blocking and is advanced only when `poll` says it is ready. A client that connects and says nothing costs a descriptor and is dropped after `BATCHD$K_CONNTMO` seconds.

Signals arrive through a self-pipe. The handler writes one byte and returns; everything else happens in the main context, where the choice of routines is not restricted. `SIGCHLD` therefore does not race with the queue.

The upper bound on the wait, `BATCHD$K_POLLMS`, exists for `/AFTER`: the arrival of an appointed time produces no event of its own, so the loop must wake and look.

Chapter 3. The Modules

3.1 Map

Module	Purpose
`batchdef.h`	Job states, the `BATCH$JOB` descriptor, capacities, prototypes of the general-purpose routines
`batchmsg.h`	Condition values of the facility, the outcome tests
`batchmsg.c`	The message table and its registration
`batchipc.h`	The protocol between the two images
`util.c`	General-purpose routines: output, time, paths, transfer
`batchd.c`	The job controller
`batch.c`	The command line utility
`batchvers.h.in`	Identification, filled in by CMake

Inclusion runs one way: `batchdef.h` draws in `batchvers.h`, `batchmsg.h` and `starlet.h`, and `batchmsg.h` draws in `utility_routines.h`. Nothing includes `batchd.c` or `batch.c`; they share nothing but `util.c` and the headers.

Order matters in one place. `utility_routines.h` declares the severity codes as an enumeration and `starlet.h` carries fallback macros of the same names under `#ifndef`. The library must therefore be included first, which `batchdef.h` arranges. Reversing it turns the enumeration into an assignment to a constant.

3.2 batchdef.h

Definitions common to both images. Three things live here and nowhere else:

- the capacities — `BATCH_K_MAXJOB`, `BATCH_K_NPARM`, `BATCH_K_PATHSZ` and the rest;
- the job states `JOB$K_*` and the retention conditions `RETAIN$K_*`;
- `BATCH$JOB`, the descriptor, described in Section 4.1.

It also states the calling convention the whole subsystem holds to, which Section 5.3 takes up.

3.3 batchmsg.h and batchmsg.c

`batchmsg.h` defines the facility number and one condition value per distinguishable outcome. The message numbers are distributed over areas with gaps between them so that adding a condition does not displace numbers already assigned:

```
  1 ..   9   general              50 ..  69   files and the journal
 10 ..  29   general-purpose      70 ..  89   communication
 30 ..  49   queue and jobs       90 .. 109   command line
                                 110 .. 129   process creation and controller
```

`batchmsg.c` holds the table itself, an array of `EMSG_RECORD`, and `msg$init` which hands it to `__util$inimsg`. Both images call `msg$init` before issuing anything. Three constraints on the table:

- it is **not** `const` — the library sorts it in place;
- a message text is written for the **FAO** engine — `!AZ`, `!SL`, `!UL`, `!SQ` — because `__util$putmsg_fao` renders it with `__util$faol`. The number of conversions in a text must equal the number of arguments every call site passes; the compiler does not check this, the format coming from the table at run time;
- the envelope `%FAC-S-IDENT` is **not** in the text. The library supplies it from the `facname` field of the descriptor, and a text carrying an envelope of its own is framed twice.

3.4 batchipc.h

The protocol. A frame is a fixed `BATCH$IPCHDR` followed by `mh_len` bytes of payload. Byte order is not converted: both ends are on one node and come from one build. The function codes are `IPC$K_*` and the selection indicators of `SHOW` are `SHOW$M_*`.

3.5 util.c

Routines both images use. None of them knows anything about jobs.

Routine	Purpose
`util$put`	Formats a line with FAO and writes it to the standard output. Every line the utility shows the user goes through here.
`util$mkfspec`	Builds a file specification with FAO, refusing to truncate. Parameters come as an array, since a variable list cannot be forwarded to `__util$faol`.
`util$fmt_time`	Renders a time as `dd-MMM-yyyy hh:mm:ss` through `__util$asctim`.
`util$parse_time`	Parses the `/AFTER` grammar. A syntax adapter over `__util$bintim`: it rewrites the specification into the form the library accepts and leaves the arithmetic to it.
`util$parse_cpulist`, `util$fmt_cpulist`	The `/CPU` list, in the notation of `taskset(1)`.
`util$mkpath`	`mkdir -p`.
`util$state_name`	Mnemonic of a job state.
`util$read_exact`, `util$write_exact`	Blocking transfer of an exact number of bytes. Used by the utility and for the journal — **not** by the controller for client connections, which must not block.

3.6 batchd.c

The controller. Its routines fall into five groups.

The queue. `s_batchd$job_alloc` and `s_batchd$job_free` are the only routines that create and destroy entries; `job_free` also removes the log file unless `/KEEP` was given. `s_batchd$job_by_entry` and `s_batchd$job_by_pid` find an entry; `s_batchd$count_executing` counts the occupied slots.

The journal. `s_batchd$journal_save` writes the whole queue to a temporary file and renames it over the previous copy, which makes the write atomic. `s_batchd$journal_load` reads it back and decides what to do with jobs that were executing when the controller stopped.

Jobs. `s_batchd$job_start` creates the process: session, signals, affinity, identity, working directory, descriptors, environment, `execv`. `s_batchd$schedule` chooses what to run and releases what has waited long enough. `s_batchd$reap` collects the status of terminated jobs and decides whether the entry stays.

Requests. `s_batchd$do_submit`, `s_batchd$do_delete`, `s_batchd$do_show` and the `IPC$K_RELEASE` branch of `s_batchd$conn_dispatch`. `s_batchd$job_selected` states the selection rule of `SHOW` in one place, because the queue is walked twice — to count and to transmit — and the two passes must agree.

Connections. `s_batchd$conn_accept`, `conn_read`, `conn_dispatch`, `conn_write`, `conn_close` and the state they keep in `BATCHD$CONN`. `s_batchd$listen` creates the listening socket; `s_batchd$log_sweep` clears the log directory of what belongs to no entry.

The one routine that is not called from the main context is `s_batchd$sighandler`: it writes a byte to the self-pipe and returns, and does nothing else.

The created process is a different world. Everything after `fork` in `s_batchd$job_start` runs with the standard streams about to become the job log, so it must not issue a message: the message would land in the log of the job. A failure there exits with `BATCHD$K_EXECFAIL`, which the reaper collects as an ordinary termination status.

3.7 batch.c

The utility. It parses, sends, and displays.

The parsing is the part with a decision in it. In DCL a qualifier begins with a slash; in Unix so does an absolute path. The CLI services of the library make no such distinction and, further, take a declared parameter from the head of the vector unconditionally. Two things are therefore done here:

- `s_batch$is_qual` classifies an argument. It is a qualifier only if the text up to the equals sign carries no slash of its own and names, unambiguously, one of the qualifiers of the table.
- `main` rebuilds the argument vector — verb, then non-qualifiers, then qualifiers — before handing it to `__cli$parse`. This is what preserves the DCL freedom of order that the services do not offer.

No verb declares a parameter, deliberately: with one declared, `batch submit /NAME=x` would hand `/NAME=x` to P1. The single parameter each command takes is picked out of the already classified arguments.

`SET ENTRY` is the one verb of two words. The CLI services carry a sub-verb table — `ENTRY` is a `CLI_VERB` under `SET`, with the qualifier table of the command — and expect the second word right after the first, so `main` puts the first argument that is not a qualifier back there before the vector is handed over. The dispatch settles the sub-verb as it settles the verb, by the address of its descriptor. `/HOLD`, `/KEEP`, `/RESTART` and `/AFTER` are negatable for it; `CLI$GET_VALUE` reports a negated qualifier as present like any other, so those four are read by `CLI$PRESENT`.

`s_batch$get_quals` draws the values out of the parsed command line. What remains there is what is peculiar to this command: splitting `/PARAMETERS`, resolving `/LOG_FILE`, converting `/AFTER`. Matching, abbreviation, ambiguity and type validation belong to the library.

3.8 The Vendored Subproject

UTILITY$ROUTINES installs itself as a `StarLet` package and is required as one; no copy of it is carried in this tree. One copy in a system is worth insisting on: the structures of that library are laid out in its headers and this subsystem embeds them - the qualifier tables of `batch.c` are `CLI_PQDESC` and the message table of `batchmsg.c` is `EMSG_RECORD` - so a second revision in one image would not announce itself.

What the subsystem uses:

From	What
`utility_routines.h/.c`	`$PUTMSG_FAO` and the message registry, `__util$deflog`, `__util$rewindlogfile`, the `__QUEUE` primitives
`starlet.h/.c`	The FAO engine, `__util$asctim`, `__util$bintim`, `__util$gettim`, `__util$add_times`
`cli_routines.h/.c`	`__cli$parse`, `__cli$get_value` and the qualifier tables

It is compiled with `-w`: it is not ours to correct, and a diagnostic from it would be indistinguishable from one of our own.

When a new revision arrives, replace the directory whole and run the tests. `test/queue.c` exists for exactly that: a defect in `$REMQENT` that was corrected once can return with the next revision, and the job queue rests upon it. It earned its keep three revisions running, each of which arrived with that defect back in place; the correction is upstream as of `V.01-09` and the directory is now vendored unaltered.

One further thing to look for after a change of revision. The library has been moving its names to a common __ prefix, module by module — the STARLET services in one revision, the CLI services in the next — so a link error naming `cli$parse` or `util$asctim` means the prefix, not a missing routine.

Corrections of ours have twice had to be carried across revisions and are now upstream, so the directory is vendored unaltered; both were of a kind worth recognizing, since neither showed itself as a compilation error. One passed a value by address where the callee had been changed to take it by value; the other returned a status whose severity had been clamped to an odd number, which under the `1 & status` convention reads as success. The library carries regression tests of its own as of this revision, and they are run along with ours.

Chapter 4. Data Structures

4.1 BATCH$JOB

The job descriptor, in `batchdef.h`. Flat, of fixed size, grouped by type so that no alignment holes appear. It is written to the journal and put on the socket exactly as it stands.

Field	Purpose
`jb_entry`	Queue entry number, unique and increasing
`jb_state`	One of `JOB$K_*`
`jb_prio`, `jb_retain`, `jb_restarts`	`/PRIORITY`, `/RETAIN`, count of restarts
`jb_pid`, `jb_uid`, `jb_gid`	The process and the identity it runs as
`jb_exit`, `jb_signal`	How it terminated
`jb_keeplog`, `jb_restart`	`/KEEP`, `/RESTART`
`jb_qtime`, `jb_after`, `jb_stime`, `jb_etime`	Queued, not before, started, ended
`jb_cpus`, `jb_queue`, `jb_name`	`/CPU`, `/QUEUE`, `/NAME`
`jb_script`, `jb_logfile`, `jb_cwd`	Absolute paths
`jb_parm`	`/PARAMETERS`, eight of them

The fields a client may not be trusted with — `jb_entry`, `jb_uid`, `jb_gid`, `jb_pid`, the timestamps — are set by `s_batchd$do_submit` and only there, whatever the client sent.

4.2 BATCHD$JOBENT and BATCHD$CONN

```
typedef struct batchd_jobent_t {
    ENTRY       je_links;   /* first: the $INSQ/$REMQ macros require it */
    BATCH$JOB   je_job;
} BATCHD$JOBENT;
```

The links are in a wrapper rather than in the descriptor. That is what keeps the journal format independent of them: what is written is `je_job`, and the queue could be replaced tomorrow without touching the file.

`BATCHD$CONN` holds what a connection needs between the steps of a transfer: the request header, the buffer, how much of it has moved, the state, the identity of the sender and the deadline. It exists because the transfer is not a single blocking call.

4.3 The Protocol

`BATCH$IPCHDR` — signature, function, condition value of the reply, payload length, record count, entry number — followed by the payload. `SUBMIT` carries a `BATCH$JOB`; `SHOW` carries a `BATCH$SHOWREQ`; `DELETE` and `RELEASE` carry nothing, the entry number being a field of the header; `SET ENTRY` carries a `BATCH$SETENT`, whose mask names the attributes it sets so that an attribute not mentioned is not touched.

The reply to `SHOW` is a header followed by `mh_count` descriptors. Every other reply is a header alone.

A condition value produced by the controller travels to the utility in `mh_sts`. The *arguments* of its message do not, so the utility reports the value and the readable text stays in the log of the controller, where it was issued in full.

4.4 The Journal

`BATCHD$JRNHDR` — signature, record size, record count, the entry number counter — followed by that many `BATCH$JOB` records.

A record is **not** the image of `BATCH$JOB`. It is a sequence of tagged values, written with the TLV routines of the library, and the header states the format of the journal rather than the size of the descriptor. A reader takes the tags it knows, passes over those it does not, and leaves at its default a field whose tag is absent. A journal therefore crosses a revision of the software in either direction: a newer BATCH reads an older journal, and an older BATCH reads a newer one as far as it understands it.

The tags, and the three rules that keep this property — a tag is never reused, never changes type, and a new field must have a default under which an old journal still describes a sensible job — are in `batchjrn.h`. Adding a field to `BATCH$JOB` now means adding a tag beside it, and nothing else.

The write goes to `journal.tmp` and is renamed over `journal.dat`. The rename is atomic within one file system, so the journal is never seen half-written. `fsync` precedes the rename, which is what makes the durability real and what makes the write expensive: see Appendix A.

Chapter 5. Conventions

The sources hold to the OpenVMS style throughout. This is not decoration: a reader who knows the convention can tell the lifetime and the origin of a name at sight.

5.1 Naming

Form	Meaning
`g_name`	Global
`l_name`	Local
`s_name`	Static — a variable, or a routine not exported from its module
`a_name`	Parameter of a routine
`BATCH_K_*`	Constant
`BATCH_M_*`	Build indicator
`BATCH$_*`	Condition value
`$MACRO`	Macro
`s_batchd$name`	Static routine: the `s_` prefix, then the module, then the name

A structure is declared with the tag carrying the suffix `_t` and the type name carrying the separator:

```
typedef struct batch_job_t { ... } BATCH$JOB;
```

5.2 Formatting

Whitesmiths: the brace of a nested block sits at the indentation of the body it opens, the brace of a routine at column zero. Tabs only, of width eight; text to column 132. Declarations at the head of a routine, without indentation, grouped by type. A single statement under `if` takes no braces. A loop counter is declared in the `for` where it can be.

Every module opens with `MODULE`, `IDENT`, `REV`, `FACILITY`, `ABSTRACT`, `AUTHOR`, `CREATION DATE` and a modification history, and every routine with a `FUNCTIONAL DESCRIPTION` block stating what it does, its input and output parameters and what it returns.

`IDENT` and `REV` appear twice in a module — in the header and in the `__IDENT__` and `__REV__` macros — and are to agree.

The identification is `X.aa-bb` in the VMS manner — a dot after the letter — and `REV` is derived from it part for part: `X.aa-bbECOcc` gives `aa.bb.cc`, so `X.01-29` gives `01.29.00`. A small correction of a fault does not raise `bb`; it is marked `ECO` instead, always of two digits, and the ECO number takes the third place — `X.01-29ECO01` gives `01.29.01`. A release tag is the identification itself, with no `v`. A module not changed since 12-NOV-2026 may still carry the older form `X-aa-bb[ECOcc]`, and the published tags keep it; the build takes both forms. Every entry of the modification history carries `RRL` between the date and the identification.

5.3 The Calling Convention

A routine returns a condition value and delivers its result through an output parameter.

```
if ( $IS_FAILURE(s_batchd$job_by_entry(a_entry, &l_ent)) )
    return  $PUTMSG_FAO(BATCH$_NOSUCHENT, a_entry);
```

The exceptions are three, and each says so in its own comment: `s_batchd$sighandler`, whose signature is imposed by the system, and the two `main` routines.

`$PUTMSG_FAO` returns the condition it was given, which is what lets a diagnostic and a return fit into one expression.

Success is tested with `$IS_SUCCESS`; a specific cause is told apart with `$MATCH_COND`, which compares the identification and disregards severity.

5.4 Diagnostics

All diagnostics go through `$PUTMSG_FAO`. The text is in the table, not at the point of the call; where one condition serves several places, the distinguishing part is a parameter of the text rather than a condition of its own.

The failure of a C RTL routine is reported where it happens, with the name of the routine, its result and `errno`:

```
if ( 0 > (l_rc = mkdir(l_work, a_mode)) && (errno != EEXIST) )
    return  $PUTMSG_FAO(BATCH$_MKDIRERR, l_work, l_rc, errno, strerror(errno));
```

Returning a condition upward and leaving the run-time library's account of the failure behind loses the only information that would have identified it.

5.5 Formatting of Output

Nothing in the subsystem calls `printf` in any of its forms, and no module includes `<stdio.h>`. Output goes through the FAO engine — `util$put` for a line to the user, `util$mkfspec` for a string — in the OpenVMS directives:

Directive	Effect
`!AZ`	Zero-terminated string
`!AC`	Counted string, an `ASC` container
`!UL`, `!SL`	Unsigned and signed decimal
`!nZL`	Zero-filled decimal of width n
`!n<...!>`	Field of fixed width, blank-filled, truncated
`!n*c`	Character repeated n times
`!%S`	`s` when the last number converted is not 1

The `!/` directive is not used. It yields the carriage return and line feed pair of OpenVMS, which a Unix stream does not expect; a line separator is written as a literal `\n`.

Chapter 6. Making a Change

6.1 Adding a Qualifier

Say the qualifier is `/NICE=n` for the utility.

1. `batchdef.h` — add the field to `BATCH$JOB`, in the group of its type. **This changes the size of the descriptor**, so an existing journal will be refused; that is expected and needs no other action.
2. `batch.c` — add the enumerator to `QUAL$K_*` and the row to `s_quals`, with the type the CLI services are to validate: `CLI$K_NUM`, `CLI$K_QSTRING`, `CLI$K_OPT` for a valueless one, `CLI$K_KWD` with a `CLI_KEYWORD` list for a set of words.
3. `batch.c` — add the field to `BATCH$CLICTX`, draw the value in `s_batch$get_quals`, place it in the descriptor in `s_batch$do_submit`.
4. `batch.c` — add it to the usage text of `s_batch$usage` and to the `ABSTRACT` of the module.
5. `batchd.c` — act upon it, usually in `s_batchd$job_start`.
6. `batch.c` — display it in `s_batch$show_full` if it is worth seeing.
7. The *User's Guide* — the qualifier entry in the command dictionary.
8. `test/smoke.sh` — a check that it is accepted and that a malformed value is refused.

Watch the abbreviations: a new qualifier sharing a leading portion with an existing one lengthens the shortest unambiguous abbreviation of both, and that is visible to the user.

6.2 Adding a Condition

1. `batchmsg.h` — take the next free number **in the right area**, and give it the severity that describes the outcome, not the mood of the caller.
2. `batchmsg.c` — add the row to `s_msgtab`, in FAO conversions and without an envelope of its own. The number of conversions in the text must equal the number of arguments every call site passes; the compiler does not check this, since the format comes from the table at run time.
3. The *User's Guide*, Chapter 6 — the mnemonic, the explanation and the user action.

The check worth running after a change to the table:

```
# every text and the message reference of the manuals carry FAO directives, not printf
# conversions, and every diagnostic goes through $PUTMSG_FAO
```

`test/smoke.sh` makes this check (SMOKE.SH X-01-26); it was written after the text of `IVENTRY` had reached a build with a `%s` the FAO engine printed as it stood. The number of arguments of a call against the directives of its text is still not checked by the tree: count them by hand.

6.3 Adding a Command

1. `batchipc.h` — a function code `IPC$K_*`, and a payload structure if the command carries one.
2. `batch.c` — a verb in `s_verbs`, a handler beside `s_batch$do_submit`, and a branch in `main`.
3. `batchd.c` — a `case` in `s_batchd$conn_dispatch` and a handler beside `s_batchd$do_submit`. **Assemble the reply in memory**; do not write to the descriptor from the handler.
4. Set `*a_dirty` if the command changed the queue, so that the loop reschedules and journals.
5. The *User's Guide* — an entry in the command dictionary.

6.4 Changing the Job Descriptor

Any change to `BATCH$JOB` alters `sizeof` and so refuses an existing journal. That is the intended mechanism and needs no version number of its own. What it does need is a line in the modification history of `batchdef.h` saying so, because the reader of a system that lost its queue after an upgrade deserves to find the reason.

Two invariants must survive the change:

- the descriptor stays flat and of fixed size — no pointers, nothing that means anything only in the address space that wrote it;
- the fields a client may not set stay set by `s_batchd$do_submit` alone.

6.5 Adding a Check to the Test

`test/smoke.sh` brings up a controller on a temporary root and exercises it through the utility. A check is three things: an action, a wait for the expected state, and `l_check`.

```
"${g_batch}" submit "${g_root}/scripts/ok.sh" /NAME=probe >/dev/null 2>&1
l_wait_for "probe" show /all
l_check $? "the job reached the queue"
```

Wait for a state; do not sleep for a while. How long a job takes depends on the load of the machine and is not known in advance. `l_wait_for` and `l_wait_gone` poll for ten seconds.

The test cleans up after itself: jobs it started are signalled by process group, since a job outlives the controller and some of them are orphaned on purpose by the restart check.

Chapter 7. The Build System

7.1 Layout

File	Purpose
`CMakeLists.txt`	Identification, common compilation options, the generated header, the StarLet package, the removal target
`src/CMakeLists.txt`	The targets and the installation
`test/CMakeLists.txt`	Registration of the tests
`etc/batchd.service.in`	Template of the systemd unit
`etc/batchd.init.in`	Template of the System V start script, for a system without systemd
`cmake/uninstall.cmake.in`	The removal script
`src/batchvers.h.in`	Template of the identification header

`batchcom` is an object library carrying `util.c` and `batchmsg.c`; both images link it, so neither is compiled twice and both are built from one and the same object.

One target stands apart from the images: `doc_pdf` re-renders the manuals from their markdown, and is built along with everything else when `BATCH_M_PDF` is set. It writes into `doc/`, beside the markdown, rather than into the build tree — the rendered manuals belong to the sources and are kept under version control with them, so that reading a manual calls for no TeX installation and the subsystem is handed over as one set. Set `BATCH_M_PDF` after changing a manual, and commit what it produced along with the change.

7.2 Compilation Options

They are held in an interface target, `batch_options`, rather than in the global `CMAKE_C_FLAGS`, so that a target may be added without repeating them:

```
-Wall -Wextra -Wno-unused-parameter -Wformat=2 -Wshadow -Wpointer-arith -Wwrite-strings
```

`BATCH_M_STRICT=ON` adds `-Werror`. **The vendored subproject is compiled with `-w`** and is exempt from all of it.

The standard is gnu11: the dollar sign in an identifier is a GNU extension and the whole naming convention rests upon it.

7.3 No Dollar in a CMake File

No identifier of a CMake file carries a dollar. The naming of the sources puts one in every name, and CMake is not the C preprocessor: a dollar within an identifier does not belong to it there. The C macros keep theirs — that is the convention of the sources, and the two languages are not the same language.

Inside a generated script — `cmake_install.cmake`, `uninstall.cmake` — the fault is worse than a matter of style. Such a script is parsed when the installation is run, and `${NAME$K_THING}` is no variable reference: under the policy in force it is a warning and an empty expansion rather than a refusal, so the fault shows itself as a service that was not started, or a directory that was not swept, while the installation reports success.

7.4 One Awkwardness

`configure_file` substitutes `@NAME@` but does not recognize a dollar sign within a variable name. The settings therefore travel to the templates through auxiliary names without the separator — `BATCH_CFG_SPOOLROOT` carries `BATCH_K_SPOOLROOT` — and the macros generated from them regain it.

Chapter 8. Testing

8.1 What Is There

Target	What it exercises
`status_convention`, `cli_features`, `tlv`	The regression tests of the StarLet package, run along with ours and governed by the same switch
`batch_queue`	The queue primitives of that library: removal at the head, at the tail, in the middle and of the last entry, with the links walked in both directions after each
`batch_smoke`	The subsystem end to end, through the utility, on a temporary root

```
ctest --test-dir build --output-on-failure
```

A test fails when its output matches `%BATCH-[EF]-`, so a check reports its own outcome and `ctest` needs to know nothing about it.

8.2 What the Smoke Test Guards

Each check exists because something was once wrong, or could plausibly go wrong unnoticed:

Check	Guards
Job retained on failure, removed on success	The retention rule, which is what makes `SHOW` useful
`/HOLD` and `/AFTER`	The holding state and its release
Job limit	The scheduler
`DELETE`	Removal of an entry and signalling of a running job
Restart of the controller	The journal
Absolute path taken as a parameter	The distinction between a qualifier and a path
Delta forms `+3`, `+:30`, `+::45`	A parser defect that the one form exercised before did not reach
Conditions issued with their mnemonics	The message facility
`/RESTART`	Requeueing versus orphaning
A silent client	That connections are served without blocking
The log sweep and `/CPU`	The sweep and the validation of a processor list
A job that starts nothing surviving a restart	An indication of change that was being discarded

The last two rows are worth reading twice: both are checks written *after* a defect reached a build, which is the only way to know a check is worth having.

Appendix A. Known Limitations

Each of these is a decision that could be revisited, and each is stated with where it lives.

The journal is written whole and flushed on every change. `s_batchd$journal_save`. Measured at a queue of 300 entries on ext4: 4579 µs for an operation that writes the journal against 29 µs for one that does not, of which the flush is some 3700 µs. Three ways out, in ascending order of work: coalesce the writes over a short interval, make the flush a matter of policy, or write the journal incrementally and compact it now and then.

One queue. The descriptor carries `jb_queue` and `SHOW` filters on it, but the job limit is common to the whole subsystem. A second queue would mean a limit per queue and a scheduler that knows about them: `s_batchd$schedule`.

The descriptor is mostly empty. 3752 bytes, of which `jb_parm` is 2048. Both the journal and the reply to `SHOW` carry it in full. Shrinking the parameter area is a user-visible reduction; packing on the wire would cost the property that there is no marshalling code.

Nice, resource limits and the control group cannot be set per job. Affinity can, since `/CPU`; the rest are inherited from the controller. `s_batchd$job_start` is where any of them would go.

`/JOB_LIMIT` defaults to 4 regardless of the machine. Now that the controller reads its own affinity mask at startup, the count of processors in it is the obvious default. `main` in `batchd.c`.

The controller wakes once a second when idle. `BATCHD$K_POLLMS`, for the sake of `/AFTER`. Computing the nearest deadline would remove the wakeups without changing the behaviour.

Diagnostics of the command line appear under `%CLI_RTNS-` rather than `%BATCH-`, since the library carries its own facility name. Left as it is deliberately: the reader can tell a complaint about the form of a command from one about its meaning.